

IFN647 Week 6 Review Questions

Solutions

Q1 Which of the following is False? And justify your answer.

- (1) A stem class is the group of words that will be transformed into the same stem by the stemming algorithm.
- (2) The most common measure for comparing words (or more generally, strings) is the edit distance, which is the number of operations required to transform one of the words into the other.
- (3) The Damerau-Levenshtein distance metric counts the maximum number of insertions, deletions, substitutions, or transpositions of single characters required to do the transformation.
- (4) A variety of techniques and data structures have been used to speed up the calculation of edit distances between the misspelled word and the words in the dictionary. These include restricting the comparison to words that start with the same letter (since spelling errors rarely change the first letter), words that are of the same or similar length (since spelling errors rarely change the length of the word), and words that sound the same.

Solution: (3)

The Damerau-Levenshtein distance metric counts the **minimum** number of insertions, deletions, substitutions, or transpositions of single characters required to do the transformation.

Q2 Noisy channel model

Consider the spelling error “**fish tink**” given by a user. We want to select the best correction of “tink” from {“tank”, “think”} words with edit distance 1. Assume all words with same edit distance have same probability and the words “tank” and “think” are quite common and have very similar priori probabilities, i.e., $P(tank) = P(think)$.

Based on the noisy channel model and these assumptions, we are likely select *tank* (rather than *think*) as the correction of *tink*. Please justify this statement.

Solution:

Based on the noisy channel model, we need to calculate $P(w|e)$, which is the probability that the correct word is w given that we can see the person wrote e .

Based on Bayes rule and language model, we have

$$P(w|e) \propto P(e|w)P(w)$$

The language model probability for a word is then calculated as a mixture of the probability that the word occurs in text and the probability that it occurs following the previous word, or

$$\lambda P(w) + (1 - \lambda)P(w|w_p)$$

where $0 \leq \lambda \leq 1$ is a parameter that specifies the relative importance of the two probabilities, and $P(w|w_p)$ is the probability of a word w following the previous word w_p .

By using the above equations, we have

$$P(tank|tink) = P(tink|tank) * P(tank) = P(tink|tank) * (\lambda P(tank) + (1 - \lambda)P(tank|fish))$$

$$P(think|tink) = P(tink|think) * P(think) = P(tink|think) * (\lambda P(think) + (1 - \lambda)P(think|fish))$$

According to the assumptions, all words with same edit distance 1 have same probability, i.e., $P(tink|tank) = P(tink|think)$. Also, the words “tank” and “think” are quite common and have very similar priori probabilities, i.e., $P(tank) = P(think)$.

But based on our common knowledge, usually we can get $P(tank|fish)) > P(think|fish))$ in a collection of documents. So, $P(tank|tink) > P(think|tink)$ is very likely.

Q3 For the give relevance judgment (Table 2) and the ranked output of IR Model 1 (Table 3).

- (1) Calculate IR Model 1’s precision at rank 6 (position 6) and precision at rank 10 (position 10).
- (2) Calculate IR Model 1’s average precision.

Table 2. Relevant Judgements

TOPIC	DocNo	Re1
101	110	1
101	111	1
101	112	0
101	113	1
101	114	1
101	115	0
101	116	1
101	117	1
101	118	0
101	119	1
101	120	0
101	121	0
101	122	0
101	123	1
101	124	0

Table 3. IR Model 1’s Output (ranked by weight)

TOPIC	DocNo	Weight
101	111	0.812
101	112	0.790
101	113	0.689
101	110	0.542
101	114	0.496
101	119	0.455
101	115	0.321
101	122	0.300
101	118	0.278
101	116	0.234
101	123	0.211
101	121	0.201
101	120	0.189
101	117	0.176
101	124	0.123

Solution:

(1)

precision at rank 6 = 83.33% (see question 2 solution)

precision at rank 10 = $|\{110, 111, 113, 114, 119, 116\}| / p = 6 / 10 = 60\%$

(2)

IR model's ranked output:

111 112 113 110 114 119 115 122 118 116 123 121 120 117 124

P at p 1 0.5 2/3 3/4 4/5 5/6 5/7 5/8 5/9 0.6 7/11 7/12 7/13 8/14 8/15

Average Precision = $(1 + 2/3 + 3/4 + 4/5 + 5/6 + 0.6 + 7/11 + 8/14) / 8$
 = $(1 + 0.667 + 0.75 + 0.8 + 0.833 + 0.6 + 0.636 + 0.571) / 8$
 = 73.21%

Q4 Fig. 1 shows the recall and precision at positions 1, 2, ..., 10 from two different queries *query 1* and *query 2*. The precision P at any standard recall level R is defined as

$$P(R) = \max\{P' : R' \geq R \wedge (R', P') \in S\}$$

where S is the set of observed (R, P) points.

(1) Calculate *MAP* of the two queries.

(2) For *query 1*, list the elements of S in curly braces.

(3) Calculate *query 1*'s precision P at any standard recall level R .

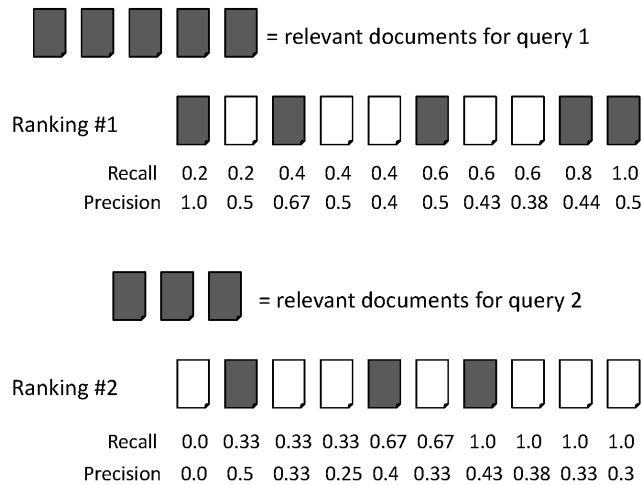


Fig 1. Recall and precision values for rankings from two different queries

(4) Assume S is represented as a list of recall-precision pairs, e.g., $S = [(0.2, 1.0), (0.2, 0.5), (0.4, 0.67), (0.4, 0.5), (0.4, 0.4), (0.6, 0.5), (0.6, 0.43), (0.6, 0.38), (0.8, 0.44), (1.0, 0.5)]$. Define a list comprehension to calculate precisions at all standard recall levels.

Solution:

(1)

average precision query 1 = $(1.0 + 0.67 + 0.5 + 0.44 + 0.5)/5 = 0.62$

average precision query 2 = $(0.5 + 0.4 + 0.43)/3 = 0.44$

mean average precision = $(0.62 + 0.44)/2 = 0.53$

(2)

$S = \{(0.2, 1.0), (0.2, 0.5), (0.4, 0.67), (0.4, 0.5), (0.4, 0.4), (0.6, 0.5), (0.6, 0.43), (0.6, 0.38), (0.8, 0.44), (1.0, 0.5)\}$

(3)

For recall level 0, we have $\{P': R' \geq 0 \wedge (R', P') \in S\} = \{1.0, 0.5, 0.67, 0.4, 0.43, 0.38, 0.44\}$

For recall level 0.1, we have $\{P': R' \geq 0.1 \wedge (R', P') \in S\} = \{1.0, 0.5, 0.67, 0.4, 0.43, 0.38, 0.44\}$

For recall level 0.2, we have $\{P': R' \geq 0.2 \wedge (R', P') \in S\} = \{1.0, 0.5, 0.67, 0.4, 0.43, 0.38, 0.44\}$
 $\max\{1.0, 0.5, 0.67, 0.4, 0.43, 0.38, 0.44\} = 1.0$

So, $P(0) = P(0.1) = P(0.2) = 1.0$

For recall level 0.3, we have $\{P': R' \geq 0.3 \wedge (R', P') \in S\} = \{0.67, 0.5, 0.4, 0.43, 0.38, 0.44\}$

For recall level 0.4, we have $\{P': R' \geq 0.4 \wedge (R', P') \in S\} = \{0.67, 0.5, 0.4, 0.43, 0.38, 0.44\}$

$\max\{0.67, 0.5, 0.4, 0.43, 0.38, 0.44\} = 0.67$

So, $P(0.3) = P(0.4) = 0.67$

For recall level 0.5, we have $\{P': R' \geq 0.5 \wedge (R', P') \in S\} = \{0.5, 0.43, 0.38, 0.44\}$

For recall level 0.6, we have $\{P': R' \geq 0.6 \wedge (R', P') \in S\} = \{0.5, 0.43, 0.38, 0.44\}$

$\max\{0.5, 0.43, 0.38, 0.44\} = 0.5$

So, $P(0.5) = P(0.6) = 0.5$

For recall level 0.7, we have $\{P': R' \geq 0.7 \wedge (R', P') \in S\} = \{0.44, 0.5\}$

For recall level 0.8, we have $\{P': R' \geq 0.8 \wedge (R', P') \in S\} = \{0.44, 0.5\}$

$\max\{0.44, 0.5\} = 0.5$

So, $P(0.7) = P(0.8) = 0.5$

For recall level 0.9, we have $\{P': R' \geq 0.9 \wedge (R', P') \in S\} = \{0.5\}$

For recall level 1.0, we have $\{P': R' \geq 1.0 \wedge (R', P') \in S\} = \{0.5\}$

$\max\{0.5\} = 0.5$

So, $P(0.9) = P(1.0) = 0.5$

(4)

```
[max([p for (r,p) in S if r >= i/10]) for i in range(11)]
```